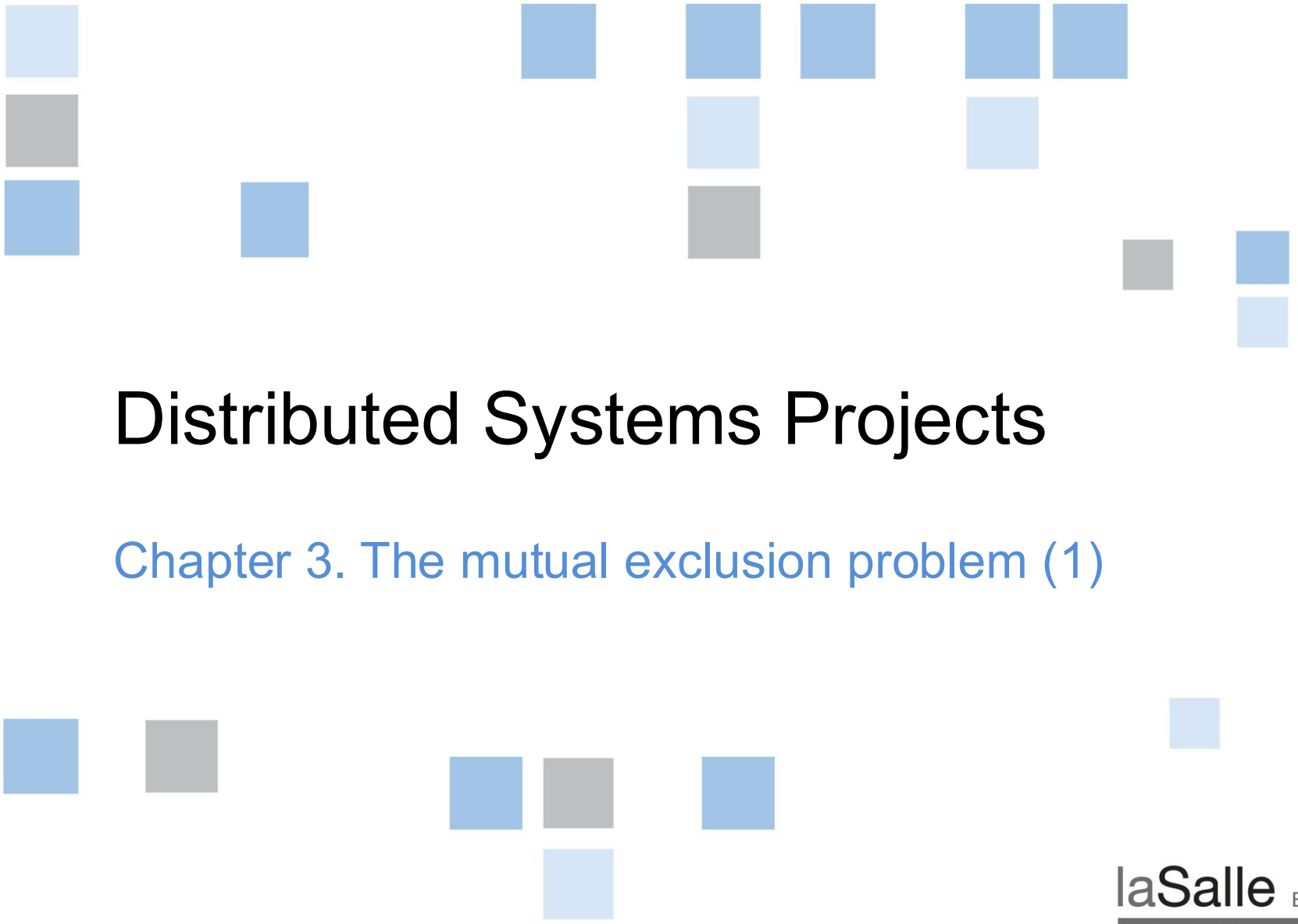


Distributed Systems Projects

Chapter 3. The mutual exclusion problem (1)

laSalle ENG
Universitat Ramon Llull

Today's agenda:

- Mutual exclusion definition
- Centralized algorithm
- Lamport algorithm
- Ricart & Agrawala algorithm

What is Mutual Exclusion and why is it needed?

Imagine you are business owner and two clients need make a payment to you bank account.

- Your two customers make simultaneous deposits of €100 into your bank account, each from a **separate** ATM.
- Both ATMs read initial amount of €1000 concurrently from your bank's cloud server
- Both ATMs add €100 to this amount (locally at the ATM)
- Both write the final amount to the server

So..... What's the problem?

- Well... You just lost €100!



The critical section problem

- Piece of code (at all processes) for which we need to ensure there is at most one process executing it at any point of time.

- Each process can call three operations.
  - **Enter** the critical section (CS)
  - **Run** the critical section code
  - **Exit** the critical section

Mutual exclusion properties

- Need to guarantee 3 properties:
  - **Safety** [essential]

    At most one process executes in CS (Critical Section) at any time
  - **Liveness** [essential]

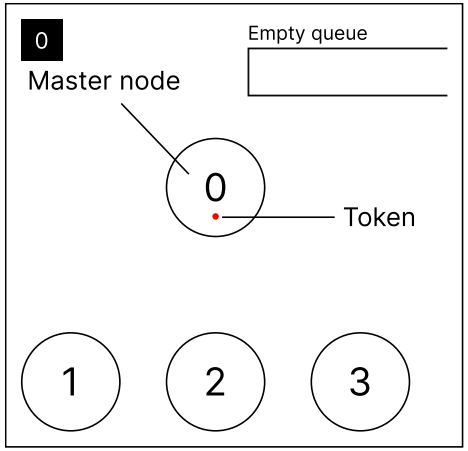
    Every request for a CS is granted eventually
  - **Ordering** [desirable]

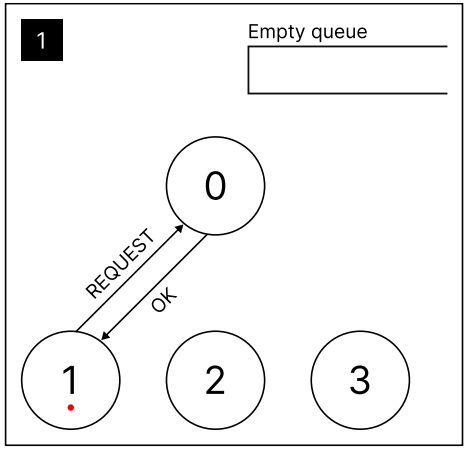
    Requests are granted in the order they were made
- Distributed systems
  - Processes can only communicate by passing messages

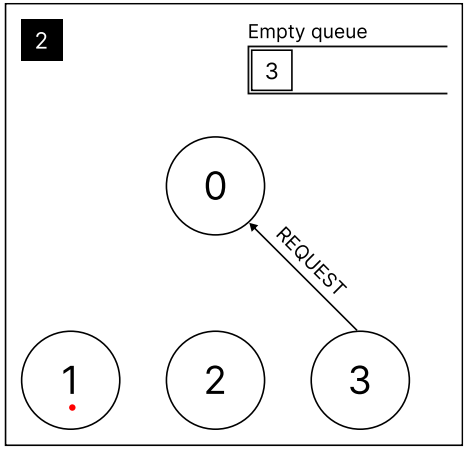
Centralized algorithm

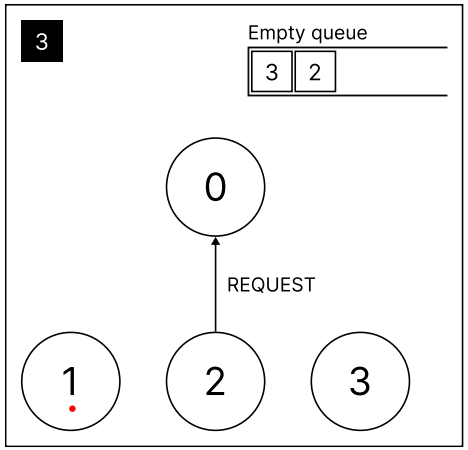
- A central coordinator (master) is needed.
- Appointed or selected (Use of leader election algorithms)
- If CS is used, the following requests are enqueued. Otherwise permission is immediately granted.
- Enqueued requests are processed when CS available again.

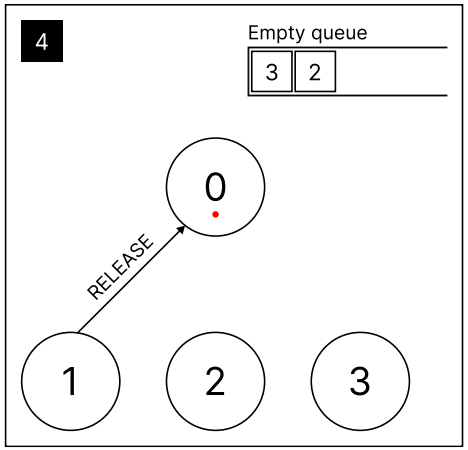
Centralized algorithm

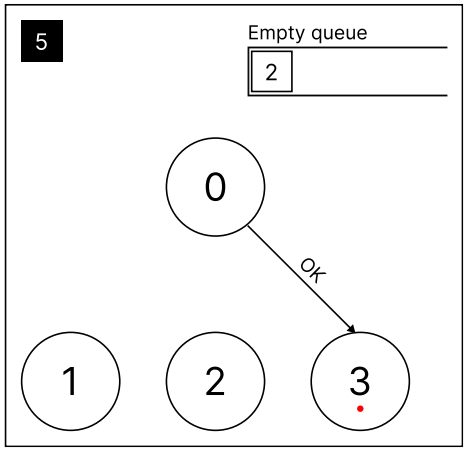












Centralized algorithm

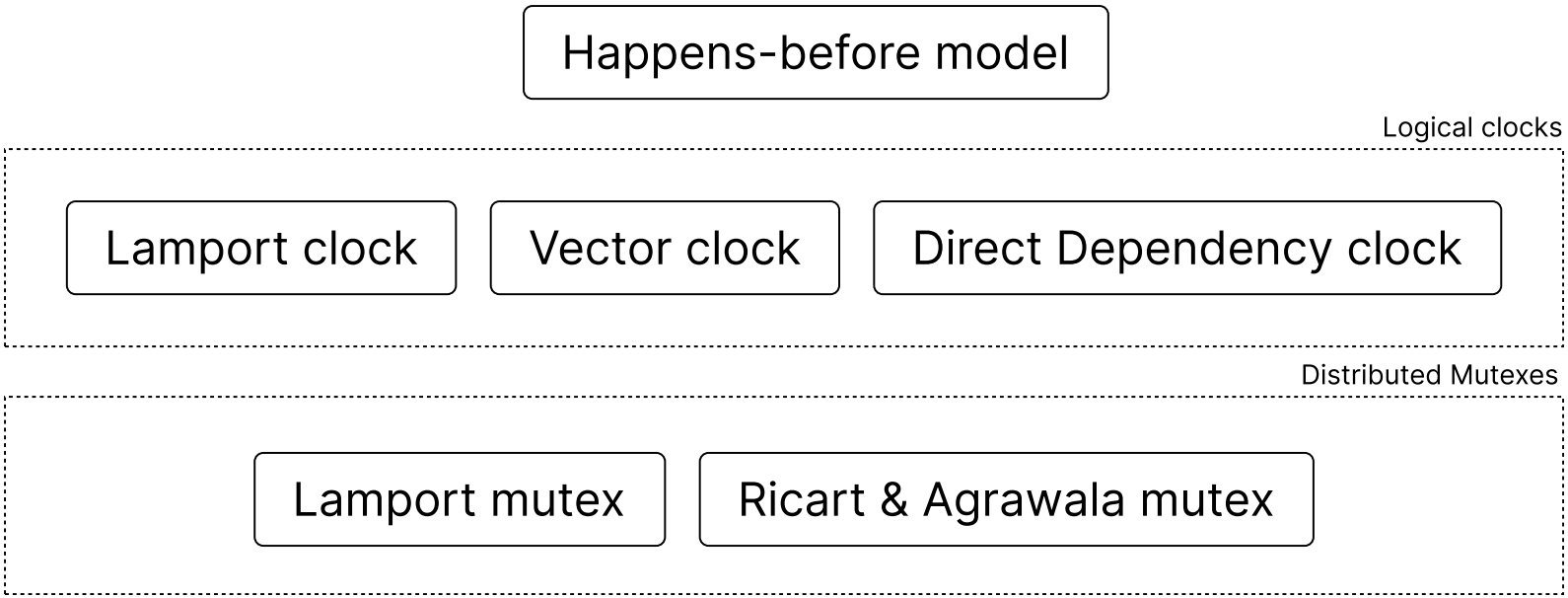
Steps of the algorithm

1. Process wants to access to the CS.
2. Sends REQUEST message to the coordinator
    1. No other process in CS: Grants permission, OKAY reply
    2. CS in use: Enqueues the request.
3. Process waits for the reply message.
4. On process exit, sends RELEASE message to the coordinator.
5. Coordinator dequeues the first deferred request and sends a OKAY reply.

The pitfall of a Centralized algorithm

The master node is

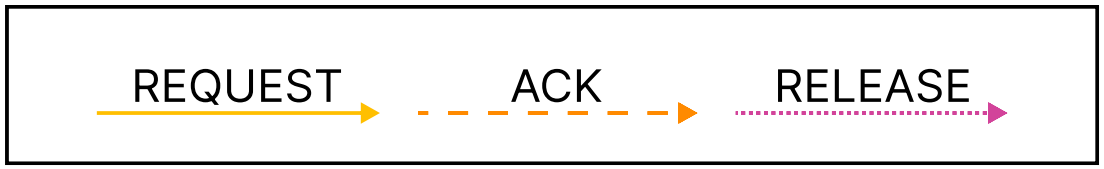
- A performance bottleneck
- Single point of failure (SPoF)


Happens-before model
Logical clocks
Lamport clock
Vector clock
Direct Dependency clock
Distributed Mutexes
Lamport mutex
Ricart & Agrawala mutex

Lamport's distributed mutex algorithm

Lamport's mutex defines **three** (3) types of messages

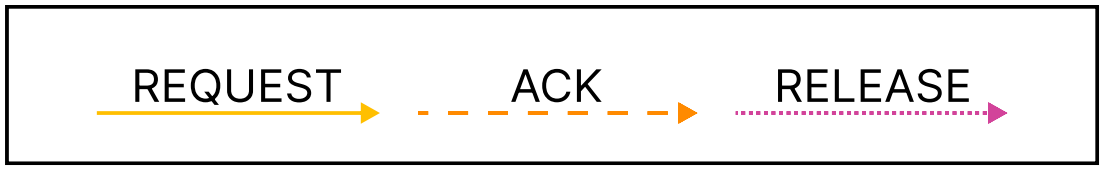
1. **Request**: Request petition when a node wants to access the Critical Section (henceforth CS).
2. **Acknowledgement**: Message sent back to the requesting node to give permission to enter the CS.
3. **Release**: Sent by the node after CS exit.



Lamport's distributed mutex algorithm

Requesting process

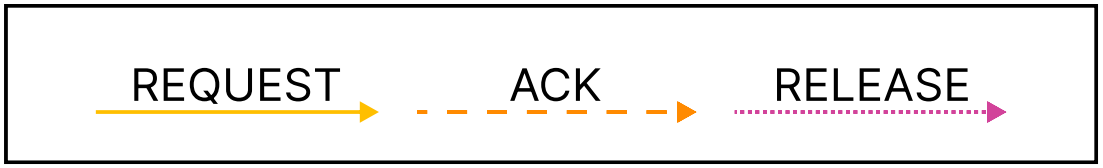
1. Pushes its own request to the Priority Queue (Ordered by timestamp)
2. Broadcast a Request message to all processes.
3. Wait for the Acknowledgement reply message from all other processes.
4. Upon exiting the CS, remove its request from the queue and send a Release message to every process.



Lamport's distributed mutex algorithm

Other process

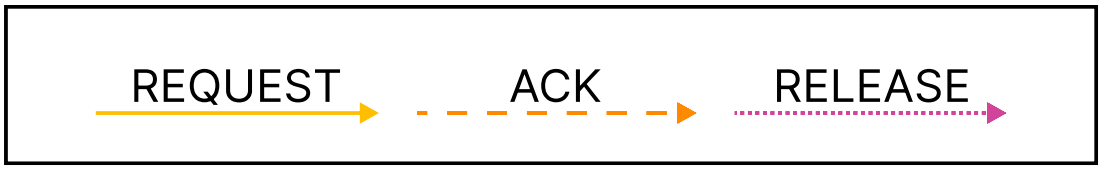
1. After receiving a Request message, it pushes the request in its own Priority Queue (ordered by timestamp) and replies with an Acknowledgement.
2. After receiving a Release message, remove the request from the queue.



Lamport's distributed mutex algorithm

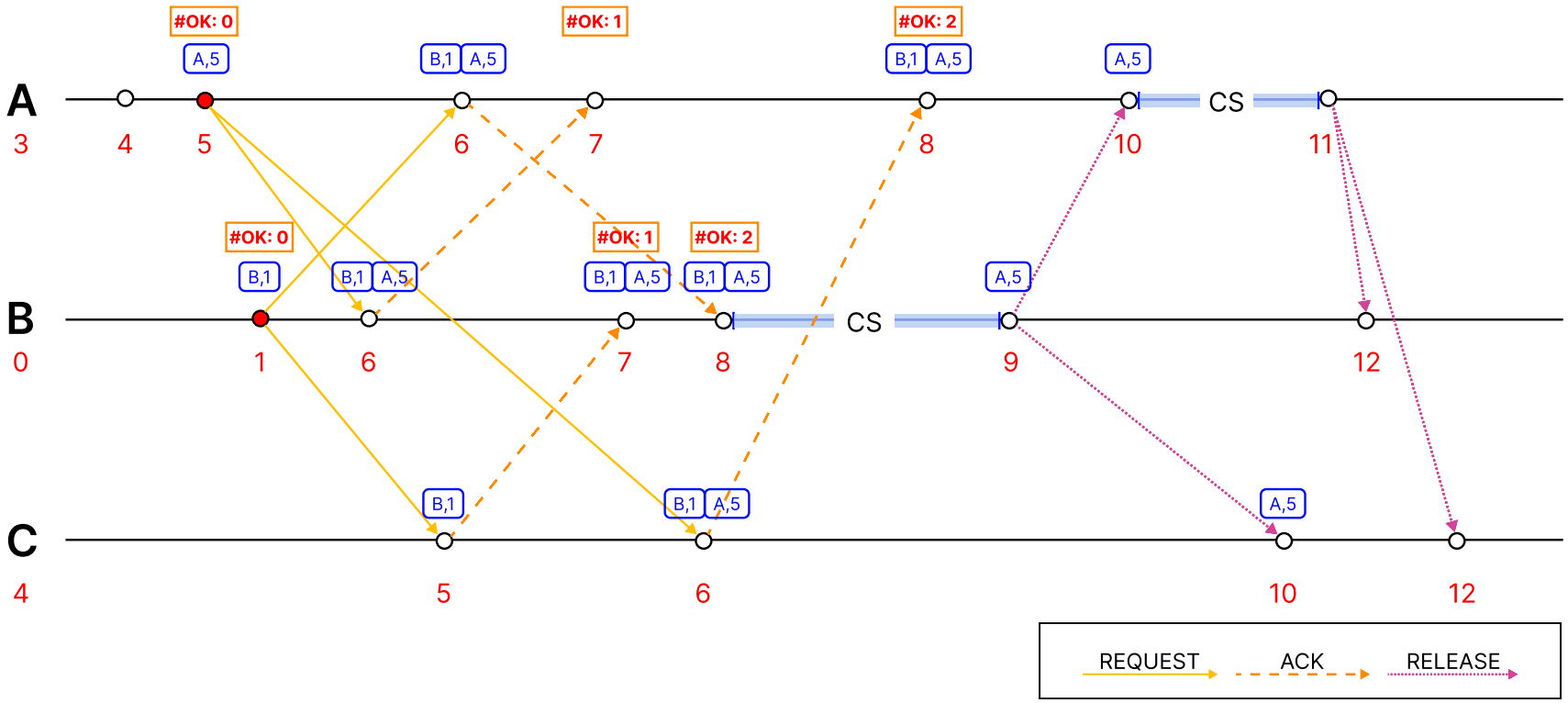
A process can access the **Critical Section** (CS) *iff* it satisfies:

1. Is interested in the Critical Section (Has broadcasted its intentions to all peers - Request)
2. Its request is at the top of the Priority Queue (Highest priority)
3. Has received an Acknowledgement message from everybody.



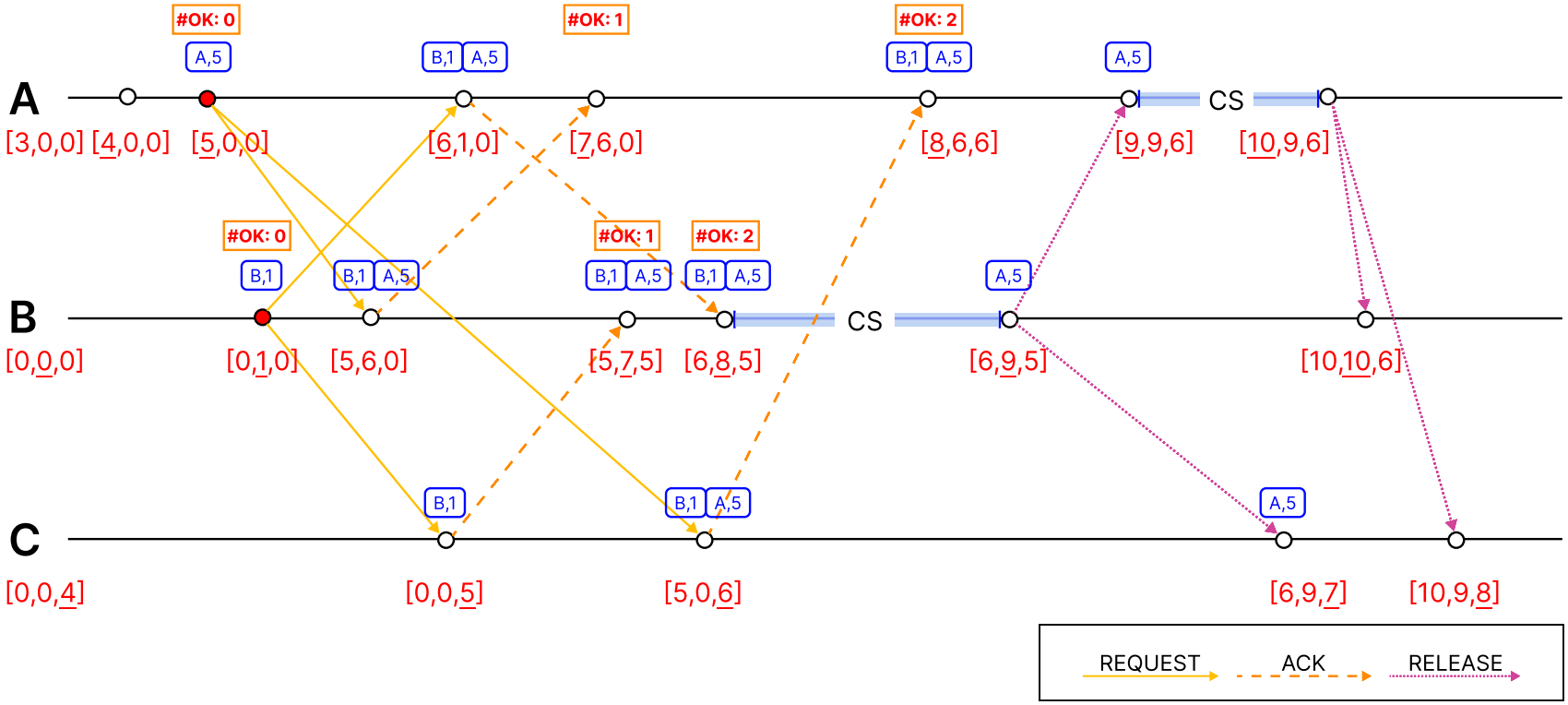
Lamport's distributed mutex algorithm

Lamport Distributed Mutual Exclusion with Lamport clock



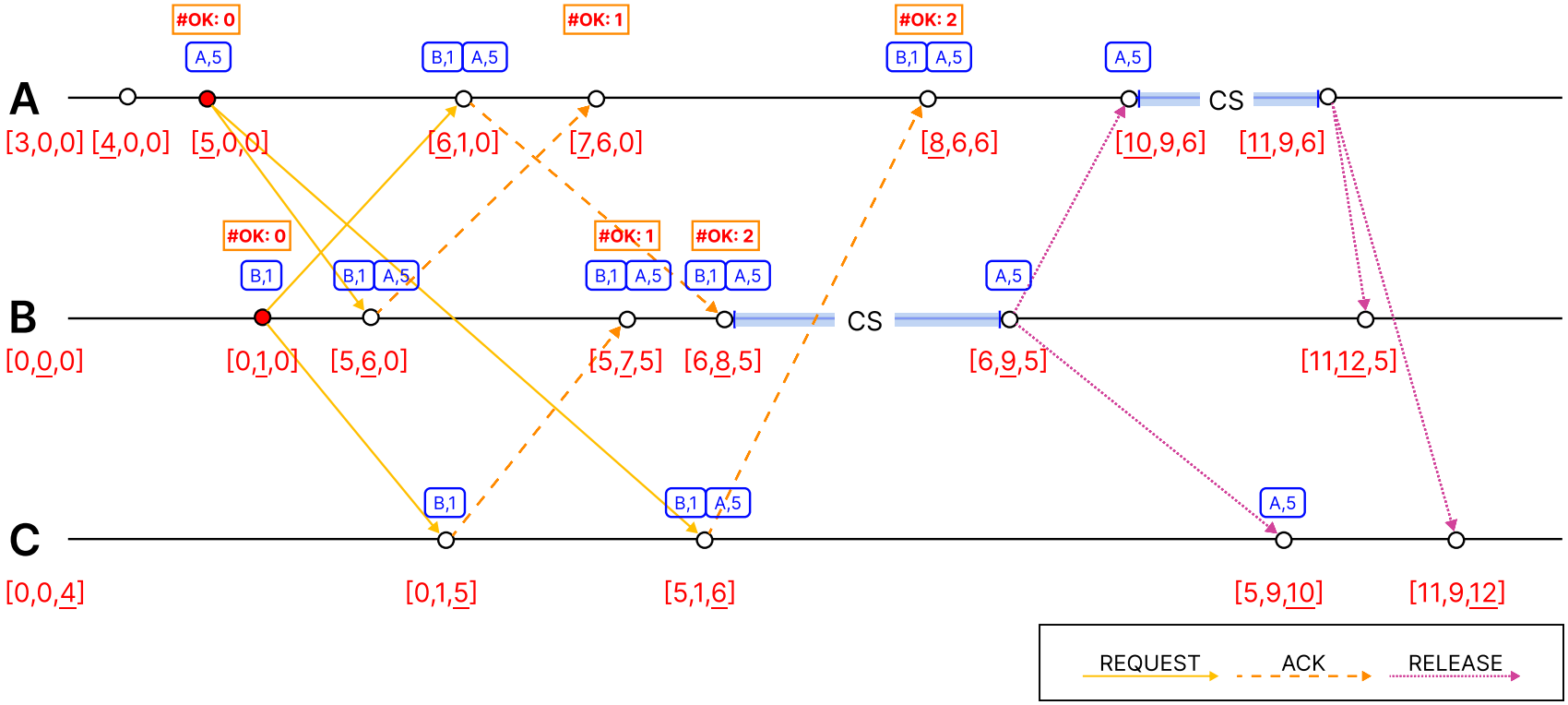
Lamport's distributed mutex algorithm

Lamport Distributed Mutual Exclusion with Vector clock



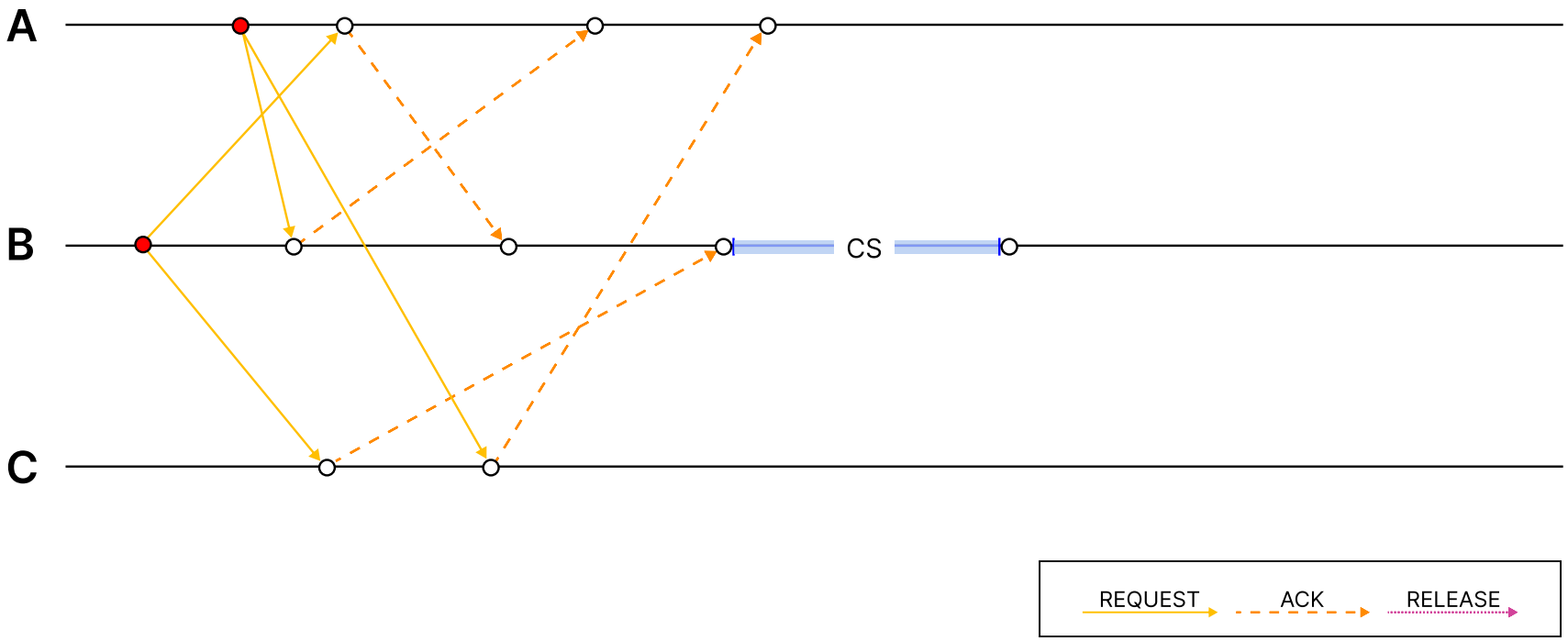
Lamport's distributed mutex algorithm

Lamport Distributed Mutual Exclusion with Direct dependency clock



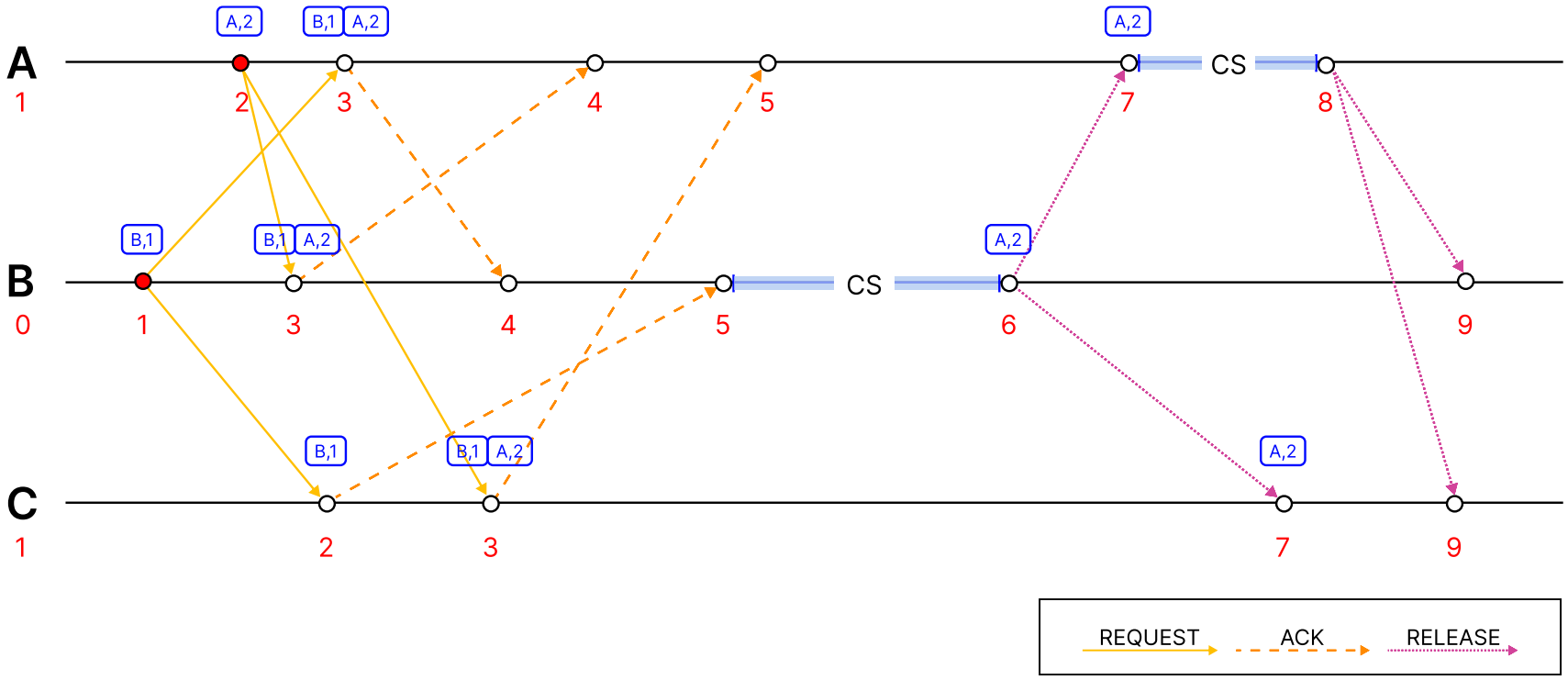
Lamport's distributed mutex algorithm exercise

Node A & Node B want to access to the CS. Finish the diagram using different logical clocks



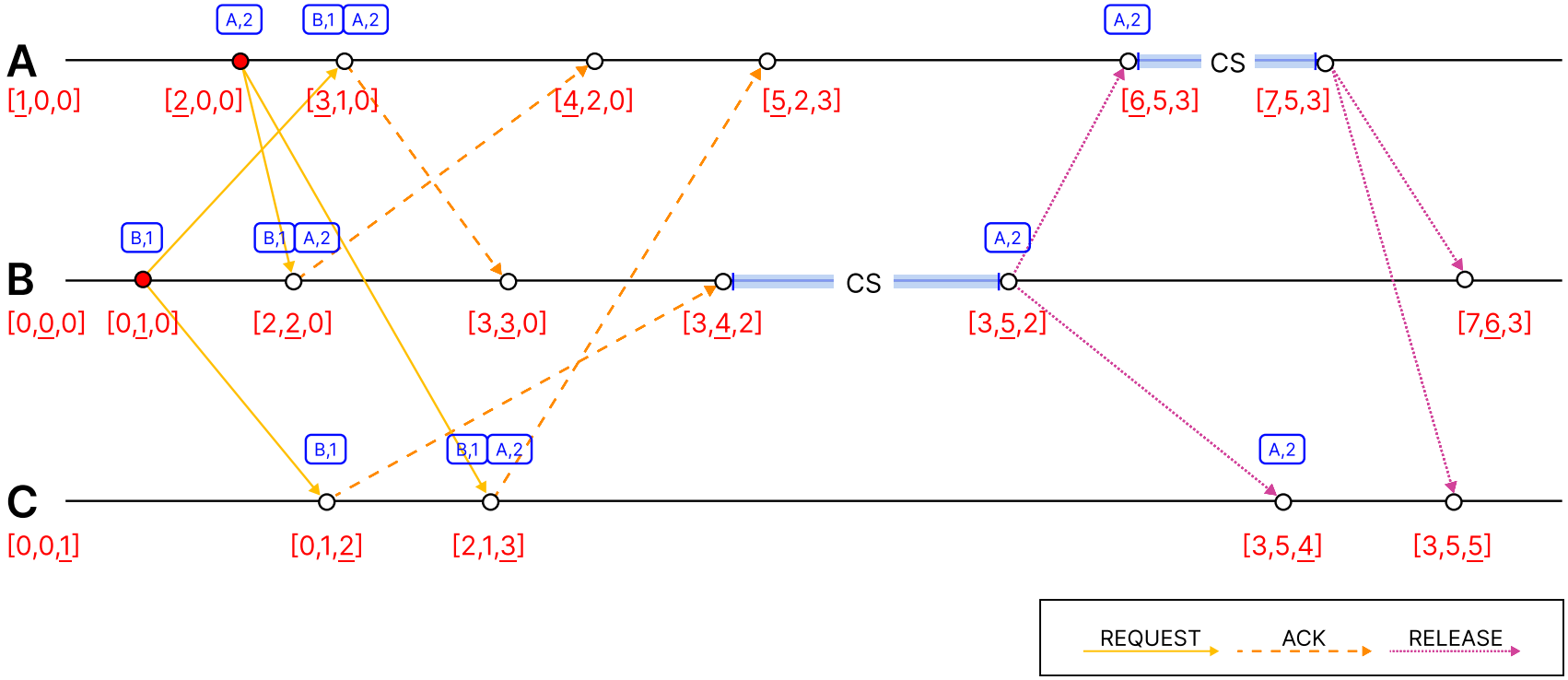
Lamport's distributed mutex algorithm exercise

Lamport Distributed Mutual Exclusion with Lamport clock



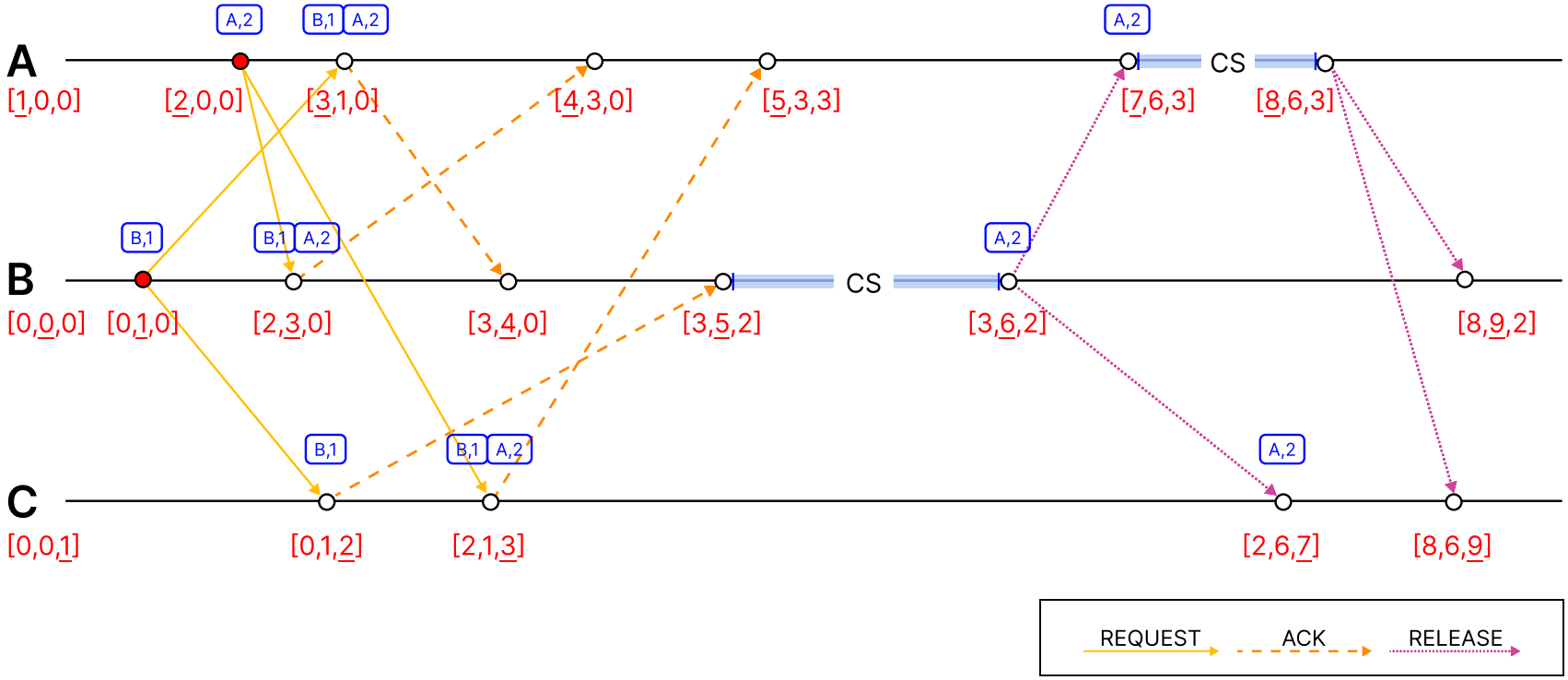
Lamport's distributed mutex algorithm exercise

Lamport Distributed Mutual Exclusion with Vector clock



Lamport's distributed mutex algorithm exercise

Lamport Distributed Mutual Exclusion with Direct dependency clock



Appendix: Centralized Mutex

```
public class CentralizedMutex extends Process implements Lock {
  // Assumes that P0 coordinates and does not request locks
  boolean haveToken;
  final int leaderID = 0;
  LinkedList pendingQ = new LinkedList<Integer>();

  public CentralizedMutex(Linker initComm) {
    super(initComm);
    haveToken = (myID == leaderID);
  }

  public synchronized void requestCS() {
    sendMsg(leaderID, "request");
    while (!haveToken) wait();
  }

  public synchronized void releaseCS() {
    sendMsg(leader, "release");
    haveToken = false;
  }

  public synchronized void handleMsg(Msg m, int src, String tag) {
    switch(tag) {
      case "request":       // Only master receives requests
        if (haveToken) {
          sendMsg(src, "okay");
          haveToken = false;
        } else {
          pendingQ.add(src);
        }
        break;
      case "release":
        if (!pendingQ.isEmpty()) {
          int pid = pendingQ.removeHead();
          sendMsg(pid, "okay")
        } else {
          haveToken = true;
        }
        break;
      case "okay":
        haveToken = true
        notify();
    }
  }
}
```

Appendix: Lamport mutex

```
public class LamportMutex extends Process implements Lock {
  DirectClock clock;
  int[] requestQ;  // Request queue (Map implementation)
  int numOkay;

  public LamportMutex(Linker initComm) {
    super(initComm);
    clock = new DirectClock(N, myID);
    requestQ = new int[N];
    numOkay = 0;
    for (int i = 0; i < N; i++)
        requestQ[i] = Symbols.Infinity;
  }

  public synchronized void requestCS() {
    clock.tick();  // Increase timer before sending event
    requestQ[myID] = clock.getValue(myID);
    broadcastMsg("request", requestQ[myID]);
    while(!okayCS()) wait();
  }

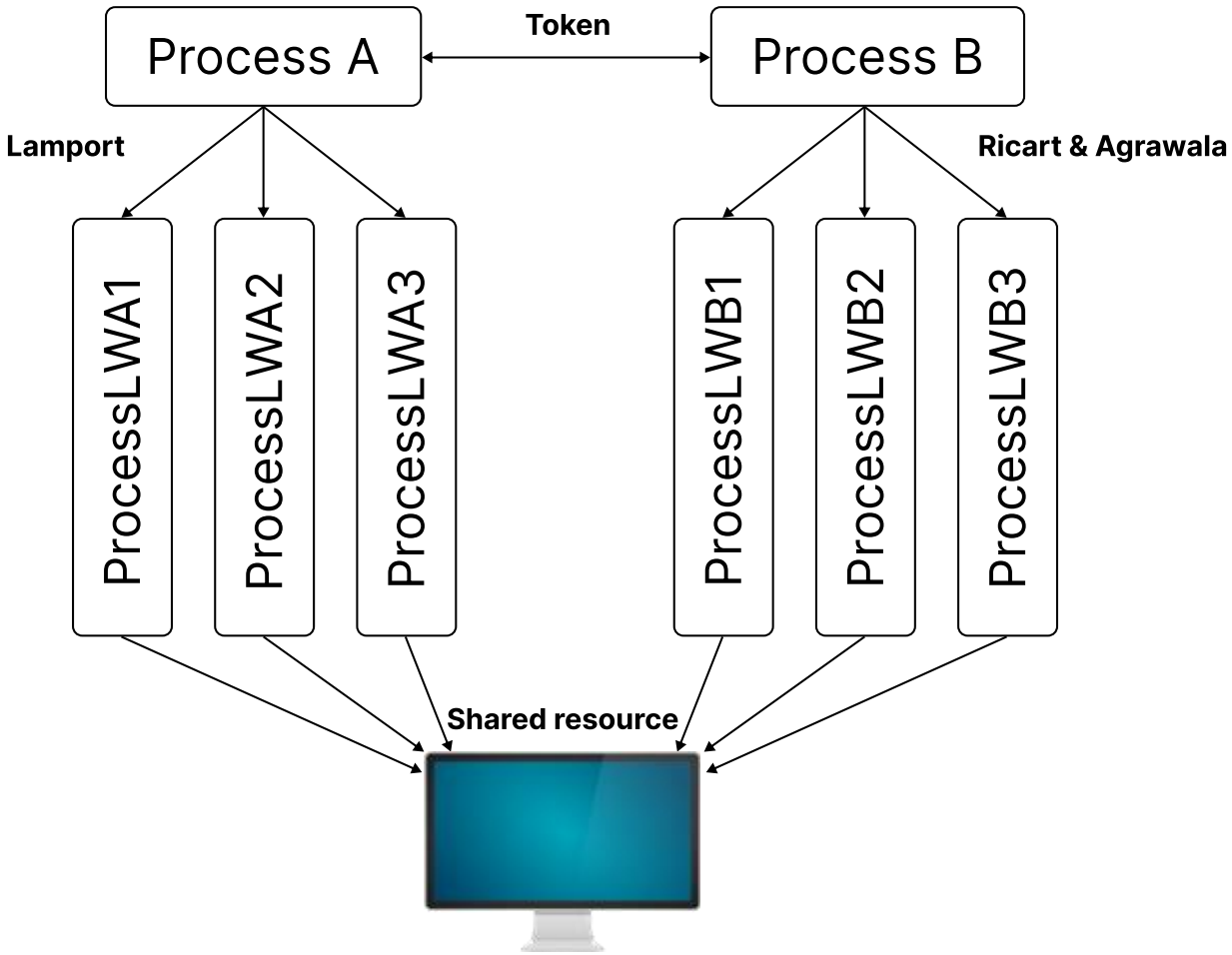
  public synchronized void releaseCS () {
    clock.tick();  // Increase timer before sending event
    requestQ[myID] = Symbol.Infinity;
    broadcastMsg("release", clock.getValue(myID));
    numOkay = 0;
  }

  private boolean okayCS() {
    if (numOkay < N - 1 ) return false;
    for(int i = 0; i < N; i++) {
      if (isGreater(requestQ[myID], myId, requestQ[i], i)) {
        return false;
      }
      if (isGreater(requestQ[myID], myId, clocks.getValue(i), i)) {
        return false;
      }
    }
    return true;
  }

  private boolean isGreater(int entry1, int pid1, int entry2, int pid2) {
  if (entry2 == Symbol.Infinity) return false;
    return ((entry1 > entry2) || ((entry1 == entry2) && (pid1 > pid2)));
  }

  public synchronized void handleMsg(Msg m, int src, String tag) {
    int timestamp = m.getMessageInt();
    clock.receiveAction(src, timeStamp);  // Sync clocks (Increase timer on reception event)
    if (tag.equals("request")) {
      requestQ[src] = timestamp;
      sendMsg(src, "ack", clock.getValue(myID));
    } else if (tag.equals("ack")) {
      numOkay++;
    }else if (tag.equals("release")) {
      requestQ[src] = Symbols.Infinity;
    }
    //okayCS may be true now
  }
}
```

Assignment


Process A
Token
Process B
Lamport
Ricart & Agrawala
ProcessLWA1
ProcessLWA2
ProcessLWA3
ProcessLWB1
ProcessLWB2
ProcessLWB3
Shared resource